

CS 4782 Homework 2

Due: 3/06/25 11:59pm on Gradescope

Late submissions accepted until 3/08/25 11:59 pm

Problem 1: Word2Vec (25 points)

1. Understanding Skip-Gram Model

The key insight behind the Skip-Gram model is that “a word is known by its neighbor words”, as we expect similar words to appear together. In Skip-Gram prediction model, we will have a center word t , and a contextual window surrounding t as $t + j$. For example, in Figure 1, we have the center word “banking”, and a contextual window with the words “turning”, “into”, “crises”, and “as”.

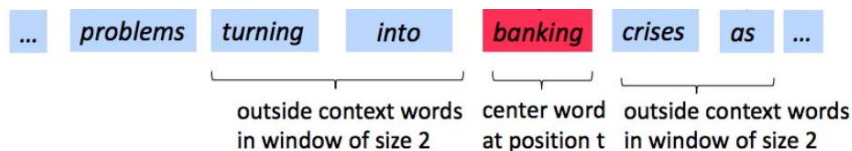


Figure 1: An example of Skip-Gram model with window size 2. Here we want to maximize the probability $P(\text{“turning”} \mid \text{“banking”})$, $P(\text{“into”} \mid \text{“banking”})$, $P(\text{“crises”} \mid \text{“banking”})$, and $P(\text{“as”} \mid \text{“banking”})$.

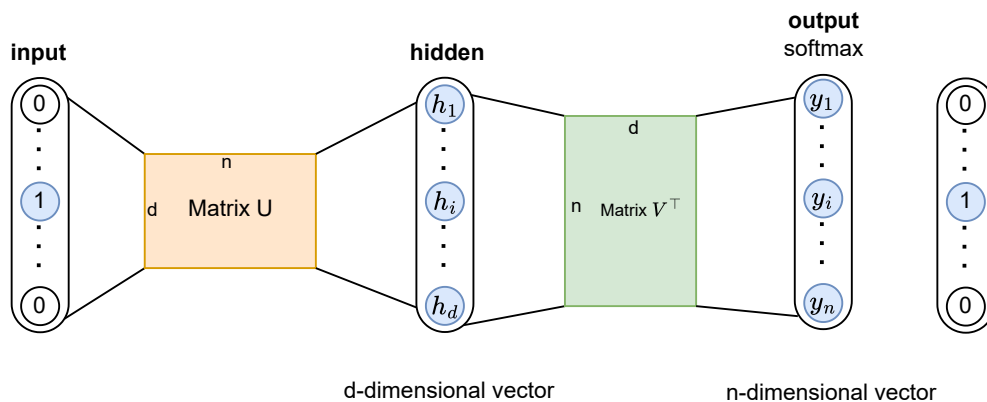


Figure 2: The Skip-Gram architecture as shown in the lecture slides. Matrices $\mathbf{U} \in \mathbb{R}^{d \times n}$ and $\mathbf{V} \in \mathbb{R}^{n \times d}$ represent the learnable vectors for the center word vectors and contextual vectors, respectively. Both $\mathbf{U}$ and $\mathbf{V}$ contain a vector for every word in the vocabulary set $\mathcal{V}$. d and n represent the dimension of the hidden word and number of words in the vocabulary set $\mathcal{V}$, respectively. Assume you have little data, so you decide to share the weights across the two layers, i.e. $\mathbf{U} = \mathbf{V}$.

The goal of the Skip-Gram algorithm is to accurately learn the probability distribution $P(x_{t+j} \mid x_t)$,

where x_t is the center word and x_{t+j} is a word inside the contextual window. We want to maximize the conditional probability as a softmax probability:

$$P(x_{t+j} | x_t) = \frac{\exp(\mathbf{U}_{:,t}^\top \mathbf{V}_{:,t+j})}{\sum_{v \in \mathcal{V}} \exp(\mathbf{U}_{:,t}^\top \mathbf{V}_{:,v})} \quad (1)$$

The above equation is retrieving specific vectors from the weight matrices $\mathbf{U}$ and $\mathbf{V}$. As a reminder, $\mathbf{U} \in \mathbb{R}^{d \times n}$, so $\mathbf{U}_{:,t}$ is retrieving the learnable vector associated with the t^{th} center word, whereas $\mathbf{V}_{:,t+j}$ is retrieving the learnable vector associated with the $t+j^{th}$ contextual word ($\mathbf{V} \in \mathbb{R}^{d \times n}$).

And the Skip-Gram “naive softmax loss” will have the following form

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} \log P(x_{t+j} | x_t) \quad (2)$$

$$= -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m, j \neq 0} (\mathbf{U}_{:,t}^\top \mathbf{V}_{:,t+j} - \log \sum_{v \in \mathcal{V}} \exp(\mathbf{U}_{:,t}^\top \mathbf{V}_{:,v})) \quad (3)$$

Here, m represents the contextual window size.

Suppose we have a vocabulary of size 5 $\{x_a, x_b, x_c, x_d, x_e\}$, and we have a $\mathbf{U} = \mathbf{V} = \begin{bmatrix} 0 & 1 & 1 & -1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 1 & 1 & -1 & 1 & -1 \end{bmatrix}$.

We are given a sequence of text as $[x_a, x_e, x_c, x_b, x_d, x_c]$

- (a) Suppose we have a center word x_c as the 3rd position in the given text, and we have a window size of 2. Please identify the words inside the contextual window. (2 points)

We know the sequence of text is $[x_a, x_e, x_c, x_b, x_d, x_c]$, and with a contextual window of size 2, we will have context words as $\{x_a, x_e, x_b, x_d\}$.

- (b) Please compute the $P(x_i | x_c)$ for all words x_i inside the contextual window of sizes 2 with the word x_c as the center word. (5 points)

$$\sum_{v \in \mathcal{V}} \exp(\mathbf{U}_{:,t}^\top \mathbf{V}_{:,v}) = \exp(-1) + \exp(0) + \exp(3) + \exp(-3) + \exp(1) = 1 + \exp(-1) + \exp(3) + \exp(-3) + \exp(1)$$

$$P(x_a | x_c) = \frac{\exp([0, 0, 1]^\top [1, -1, -1])}{\sum_{v \in \mathcal{V}} \exp(\mathbf{U}_{:,t}^\top \mathbf{V}_{:,v})} = \frac{-1}{1 + \exp(-1) + \exp(3) + \exp(-3) + \exp(1)}$$

$$P(x_e | x_c) = \frac{\exp(1)}{1 + \exp(-1) + \exp(3) + \exp(-3) + \exp(1)}$$

$$P(x_b | x_c) = \frac{\exp(0)}{1 + \exp(-1) + \exp(3) + \exp(-3) + \exp(1)}$$

$$P(x_d | x_c) = \frac{\exp(-3)}{1 + \exp(-1) + \exp(3) + \exp(-3) + \exp(1)}$$

- (c) What are the limitations of using the naive softmax loss shown in Equation 3 in a large text dataset in terms of computational efficiency? Please justify your answer. (5 points)

No, the naive softmax loss in Equation 3 would compute the inner product **over the entire vocabulary set** for every single center word in the dataset. This is too computationally expensive when we have a large text dataset.

- (d) **Negative sampling** is commonly used as an alternative to the naive softmax loss. The idea of negative sampling loss is that for each (context word, center word) pair, we will create a random subset $\mathcal{V}_S = \{x_1, \dots, x_k\} \subset \mathcal{V}$ of size k without replacement, and each word in $\mathcal{V}_S$ **is not** inside the contextual window of the center words.

We can then change the problem of maximizing the conditional probability in Equation 1 into a **binary classification problem** between predicting whether a certain word is a contextual word. In this case, the probability of an “*is contextual word*” relationship between a word x_{t+j} and x_t is defined as $\sigma(\mathbf{U}_{:,t}^\top \mathbf{V}_{:,t+j})$ where σ is the sigmoid function.

In particular, we want to find $\mathbf{U}$ and $\mathbf{V}$ to jointly maximize the probability of a correct contextual word and minimize the probability of a false/negative contextual word as

$$P(x_{t+j}|x_t) \prod_{i=1}^k (1 - P(x_i|x_t)) \quad (4)$$

Then, we will have negative sampling loss as

$$\mathcal{L}_{\text{NS}} = - \left(\prod_{t=1}^T \prod_{-m < j < m, j \neq 0} \left(P(x_{t+j}|x_t) \prod_{k=1}^K (1 - P(x_k|x_t)) \right) \right)^{1/T} \quad (5)$$

- i. Please simplify $-\log(-\mathcal{L}_{\text{NS}})$ for the Skip-Gram model using Equation 5. (Hint: $1 - \sigma(x) = \sigma(-x)$) (7 points)

$$\begin{aligned} -\log(-\mathcal{L}_{\text{NS}}) &= -\frac{1}{T} \log \left(\prod_{t=1}^T \prod_{-m < j < m, j \neq 0} \left(P(x_{t+j}|x_t) \prod_{k=1}^K (1 - P(x_k|x_t)) \right) \right) \\ &= -\frac{1}{T} \sum_{t=1}^T \sum_{-m < j < m, j \neq 0} \log \left(P(x_{t+j}|x_t) \prod_{k=1}^K (1 - P(x_k|x_t)) \right) \\ &= -\frac{1}{T} \sum_{t=1}^T \sum_{-m < j < m, j \neq 0} \left(\log P(x_{t+j}|x_t) + \sum_{k=1}^K \log(1 - P(x_k|x_t)) \right) \\ &= -\frac{1}{T} \sum_{t=1}^T \sum_{-m < j < m, j \neq 0} \left(\log \sigma(\mathbf{U}_{:,t}^\top \mathbf{V}_{:,t+j}) + \sum_{k=1}^K \log(1 - \sigma(\mathbf{U}_{:,k}^\top \mathbf{V}_{:,t+j})) \right) \\ &= -\frac{1}{T} \sum_{t=1}^T \sum_{-m < j < m, j \neq 0} \left(\log \sigma(\mathbf{U}_{:,t}^\top \mathbf{V}_{:,t+j}) + \sum_{k=1}^K \log(\sigma(-\mathbf{U}_{:,k}^\top \mathbf{V}_{:,t+j})) \right) \end{aligned}$$

- ii. The idea of negative sampling is prevalent in unsupervised learning in general. Here please identify at least 2 advantages of negative sampling loss over the naive softmax loss in Skip-Gram model. (Hint: let's think about the computational efficiency / separation in the embedding space by negative sampling). (6 points)

- Negative sampling only needs a subset of k words per each step to compute the loss, while naive softmax loss needs to loop over all vocabulary for a single (context word, center word) pair.
- Negative sampling can make the word embeddings cluster better by encouraging co-occurred words moving together and non-co-occurred words stay apart at the same time. It would generate word embeddings that learn the distance and similarity relationship in the embedding space better.

Problem 2: Exploring the Mathematics Behind Attention (25 points)

1. We saw in class that $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$. Let $A = QK^\top$. In terms of K 's key vectors and Q 's query vectors (both of which are row vectors), what is $A_{i,j}$? (3 points)

$$A_{i,j} = \sum_{k=1}^{d_k} Q_{i,k} K_{k,j}^\top = \sum_{k=1}^{d_k} Q_{i,k} K_{j,k} = Q_i \cdot K_j.$$

Thus $A_{i,j}$ is the dot product of the i th key vector and j th query vector.

2. Assume that you have random vectors $x, y \sim \mathcal{N}(\mu, \sigma^2 I)$ with $x, y, \mu \in \mathbb{R}^d$, $\sigma > 0$, and $I \in \mathbb{R}^{d \times d}$. What is $E[x^\top y]$? (4 points)

$$E[x^\top y] = E\left[\sum_{i=1}^d x_i y_i\right] = \sum_{i=1}^d E[x_i y_i] = \sum_{i=1}^d E[x_i] E[y_i] = \sum_{i=1}^d \mu_i \cdot \mu_i = \sum_{i=1}^d (\mu_i)^2 = |\mu|^2$$

3. Let $\mu = 0$ and $\sigma = 1$. What is $\text{Var}[x^\top y]$? (Hint: You might find some of these [properties](#) useful) (10 points) This first solution uses identities $\text{Var}(XY) = [E(X)]^2 \text{Var}(Y) + [E(Y)]^2 \text{Var}(X) + \text{Var}(X) \text{Var}(Y)$ and $\text{Var}\left(\sum_i^n a_i X_i\right) = \sum_{i=1}^n a_i^2 \text{Var}(X_i) + 2 \sum_{1 \leq i < j \leq n} a_i a_j \text{Cov}(X_i, X_j)$.

$$\begin{aligned} \text{Var}[x^\top y] &= \text{Var}\left[\sum_{i=1}^d x_i y_i\right] = \sum_{i=1}^d \text{Var}[x_i y_i] + 2 \sum_{1 \leq i < j \leq n} \text{Cov}(x_i, x_j) = \sum_{i=1}^d \text{Var}[x_i y_i] \\ &= \sum_{i=1}^d E[x_i]^2 \text{Var}[y_i] + E[y_i]^2 \text{Var}[x_i] + \text{Var}[x_i] \text{Var}[y_i] = \sum_{i=1}^d \text{Var}[x_i] \text{Var}[y_i] = \sum_{i=1}^d 1 = d \end{aligned}$$

Alternate solution:

$$\begin{aligned} \text{Var}[x^\top y] &= E[(x^\top y - E[x^\top y])^2] = E[(x^\top y)^2] = E\left[\sum_{i=1}^d x_i y_i \sum_{j=1}^d x_j y_j\right] = E\left[\sum_{i=1}^d \sum_{j=1}^d x_i y_i x_j y_j\right] \\ &= \sum_{i=1}^d \sum_{j=1}^d E[x_i y_i x_j y_j] = \sum_{i=1}^d E[x_i^2 y_i^2] + \sum_{i=1}^d \sum_{\substack{j=1 \\ j \neq i}}^d E[x_i y_i x_j y_j] = \sum_{i=1}^d E[x_i^2] E[y_i^2] + \sum_{i=1}^d \sum_{\substack{j=1 \\ j \neq i}}^d E[x_i] E[y_i] E[x_j] E[y_j] \\ &= \sum_{i=1}^d 1 \cdot 1 + \sum_{i=1}^d \sum_{\substack{j=1 \\ j \neq i}}^d 0 \cdot 0 \cdot 0 \cdot 0 = d \end{aligned}$$

Note that $E[x_i^2] = E[y_i^2] = 1$ since $\sigma^2 = \text{Var}[x_i] = E[x_i^2] - E[x_i]^2 = E[x_i^2]$. The same goes for y_i .

4. Assuming that the key and query vectors in K and Q similarly have 0 mean and covariance matrix I . Using your answers to previous parts, what is $\text{Var}[A_{i,j}]$? (2 points)

The key and query vectors are in the same form as x and y from question (2). Question (1) showed that $A_{i,j} = Q_i \cdot K_j$, so $\text{Var}[A_{i,j}] = \text{Var}[Q_i \cdot K_j] = d$.

5. In the formula for attention, what effect does dividing by $\sqrt{d_k}$ have on the variance of each element of $\frac{QK^\top}{\sqrt{d_k}}$? Why might this be useful? (6 points)

Dividing by $\sqrt{d_k}$ makes the variance 1. $\text{Var}[cZ] = c^2 \text{Var}[Z]$ so $\text{Var}\left[\frac{QK_{i,j}^\top}{\sqrt{d_k}}\right] = \frac{\text{Var}[QK_{i,j}^\top]}{d_k} = \frac{d_k}{d_k} = 1$. The softmax function is used on the output of $\frac{QK^\top}{\sqrt{d_k}}$, values that are considerably larger than others will become even larger after a softmax is applied. This will lead to a very small gradient for many of the weights. Setting the variance to 1 helps reduce the probability of this occurring.

Problem 3: Transformers vs RNNs (15 points)

Compare and contrast transformers with RNNs for the following aspects:

1. Briefly explain why vanishing/exploding gradients appear in RNNs during optimization. (2 points)
During backpropagation through time, the gradient of the loss function with respect to the weights, $\frac{\partial L}{\partial W}$, involves repeated multiplications of the weight matrix W across multiple time steps. If the largest singular value of W is greater than 1, the gradients can exponentially grow, leading to the exploding gradient problem. Conversely, if it is less than 1, the gradients shrink exponentially, causing the vanishing gradient problem.
2. RNNs aren't great at capturing long-term dependencies; explain how attention helps transformers address these issues. (3 points)
Transformers address this issue through the attention mechanism which allows them to selectively attend to relevant tokens across the input sequence. This allows for the capture of long-range dependencies.
3. Discuss the time complexity of both architectures with respect to sequence length and how they scale with longer sequences. (5 points)
The self-attention mechanism in transformers has a time complexity of $O(L^2)$. The quadratic dependency on L means that transformers are slow when used on longer sequences (and is why optimizations such as sparse attention exist). RNNs have a time complexity of $O(L)$. Although this is smaller than $O(L^2)$, their sequential nature doesn't allow for the simple parallelization of computations, unlike transformers.
4. Applications: Provide examples of tasks where transformers are particularly effective compared to RNNs, and vice versa. (5 points)
Transformers excel in tasks requiring capturing contextual relationships and dependencies over long sequences, such as machine translation and image captioning. Examples include BERT for natural language understanding and GPT for language generation.

RNNs are suitable for tasks where sequential information is crucial, but dependencies are not too long-range. Examples of tasks include time series prediction and speech recognition.

Problem 4: Read a Foundational Paper (16 points)

Read [Attention is All You Need](#) (2017), and answer the following questions. This paper is foundational for nearly all of deep learning, and the following questions will make sure that you understand the key methods and results of the paper. **Please answer all questions in three sentences or less.** The point of the questions is simply to check your comprehension.

1. What is the main motivation for using self-attention instead of recurrence or convolution? (4 points)
See section 4 of the paper.
2. How does the transformer model take into account the relative positions of tokens? What do they mention as other possible ways to do so? (4 points)
See section 3.5 of the paper.
3. Briefly compare the transformer model's performance and efficiency to other state of the art models. Cite a specific table or score in your answer. (As a note, the BLEU scoring method, or Bilingual

Evaluation Understudy score, compares generated sentences to reference sentences using n-grams.) (4 points)

Answer should contain proper comparison of scores, pointing to at least one statistic.

4. Which part of the paper interested you most? Briefly describe something you found exciting about the methods, results, or anything else. (4 points)

Any reasonably thought out answer is acceptable.

Problem 5: Self Attention by Hand (19 Points)

In this problem, we'll have you compute one pass through of single-head attention on a given matrix X , which contains 3-dimensional tokens representing words (for simplicity, one token corresponds to one word here).

We are going to be looking at the phrase, "dogs and cats are cute." Row X_1 represents the word "dogs," X_2 represents the word "and," and so on. (Note: For the simplicity of this problem, we will assume that this matrix already has positional embeddings included.)

$$X = \begin{bmatrix} 2 & 1 & 0 \\ 3 & -1 & 0 \\ 1 & 1 & 0 \\ -3 & -1 & 0 \\ 4 & 0 & 1 \end{bmatrix}$$

Let's assume that the weights of the key, query and value matrices are initialized randomly as below:

$$W_Q = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, W_K = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, W_V = \begin{bmatrix} 0.2 \\ 0.4 \\ 0.3 \end{bmatrix}$$

1. Fill in the values of matrix $\text{softmax}(\frac{QK^T}{\sqrt{d_k}})$ below, and show your work along the way. (8 points)

$$\text{softmax}(\frac{QK^T}{\sqrt{d_k}}) =$$

	dogs	and	cats	are	cute
dogs					
and					
cats					
are					
cute					

First, let's calculate Q and K from our input embedding matrix X and our weight matrices W_Q and W_K :

$$Q = XW_Q = \begin{bmatrix} 2 & 1 & 0 \\ 3 & -1 & 0 \\ 1 & 1 & 0 \\ -3 & -1 & 0 \\ 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

$$K = XW_K = \begin{bmatrix} 2 & 1 & 0 \\ 3 & -1 & 0 \\ 1 & 1 & 0 \\ -3 & -1 & 0 \\ 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \\ 0 \end{bmatrix}$$

Now, let's multiply QK^T and divide element-wise by $\sqrt{d_k}$:

$$QK^T = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & -1 & 1 & -1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 1 & -1 & 0 \end{bmatrix}$$

Since our key vector is 1 dimensional, and thus $d_k = 1$, we remain with:

$$\frac{QK^T}{\sqrt{d_k}} = QK^T$$

Finally, let's take the *softmax* of each row, and fill in the table correspondingly.

$$\text{softmax}(QK^T) =$$

	dogs	and	cats	are	cute
dogs	0.2	0.2	0.2	0.2	0.2
and	0.2	0.2	0.2	0.2	0.2
cats	0.2	0.2	0.2	0.2	0.2
are	0.2	0.2	0.2	0.2	0.2
cute	0.379	0.051	0.379	0.051	0.139

- Write a sentence about something you notice in the table above. What do different rows and columns represent? What is "paying attention" to what? (3 points)

Here, we see that along dimensions "dogs" and "cats," "cute" has higher values, showing that cute *pays attention* to dogs and cats. We also see from looking at the key and query matrices we calculated that the query matrix gives large positive values to cute (an adjective), while the key matrix gives large positive values to nouns.

- Now, find Attention(Q, K, V). (4 points)

First, let's find $V = XW_V$.

$$V = XW_V = \begin{bmatrix} 2 & 1 & 0 \\ 3 & -1 & 0 \\ 1 & 1 & 0 \\ -3 & -1 & 0 \\ 4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.2 \\ 0.4 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.2 \\ 0.6 \\ -1 \\ 1.1 \end{bmatrix}$$

Now, let's multiply our $\text{softmax}(QK^T)$ matrix from part 1 by this V.

$$\text{softmax}(QK^T)V = \begin{bmatrix} 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0.379 & 0.051 & 0.379 & 0.051 & 0.139 \end{bmatrix} \begin{bmatrix} 0.8 \\ 0.2 \\ 0.6 \\ -1 \\ 1.1 \end{bmatrix} = \begin{bmatrix} 0.34 \\ 0.34 \\ 0.34 \\ 0.34 \\ 0.643 \end{bmatrix}$$

Also, check out these two Medium articles for more information about these computations: [source 1](#), [source 2](#).

- Let's assume we are applying look-ahead masking to enforce that our model does not "cheat" by looking ahead to future tokens. Do we apply this masking *before* or *after* we do the softmax, and why? (4 points)

- (a) We apply the look-ahead mask to make sure that the decoder does not "cheat" by looking ahead to future tokens in a sequence. Typically, these masks have negative infinity values along the upper triangular, and 0s everywhere else.
- (b) We apply the mask within the *softmax* to ensure that the probabilities calculated by *softmax* still sum to 1, and the values for tokens we haven't seen yet are forced to (near) zero.